

OpenClaw

Setup Guide

Model Routing & Optimization

Complete Step-by-Step Instructions

Before You Begin: How to Use This Guide

This guide walks you through setting up OpenClaw from start to finish. Every single step is explained — even the basic ones. You do not need any technical background to follow it.



Good to Know

Read every step in order. Do not skip ahead.

Gray boxes contain commands or code — copy and paste them exactly as written. Do not retype them.



Orange boxes are warnings. Stop and read them carefully before continuing.



Blue boxes are helpful tips. They explain why something works the way it does.



Green boxes tell you what success looks like. If you don't see what's described, stop and recheck.

This guide is written for a Linux VPS (a remote computer you access over the internet) or any Mac or Linux terminal. You do not need a screen or desktop on your server — everything is done by typing commands.

Prerequisites — What You Need Before You Start

Before touching anything, confirm you have all of the items below. If you're missing any of them, stop here and get them first.

Required Items

- Your VPS login details — the **server address (hostname or IP)**, your **username**, and either a **password** or a **key file**
- OpenClaw already installed on your VPS and **launched at least once** — so the config file exists at `~/.openclaw/openclaw.json`

- An Anthropic API key — get one at console.anthropic.com
- At least one backup API key: **OpenAI** (platform.openai.com), **Google Gemini** (aistudio.google.com), or **DeepSeek** (platform.deepseek.com)

Optional — Recommended if You Use VS Code

- Visual Studio Code — code.visualstudio.com
- The **Remote - SSH** extension installed inside VS Code

Important — Read Before Continuing

An 'API key' is a secret password that lets OpenClaw talk to AI services like Claude, ChatGPT, or Gemini.

Never share your API keys with anyone. Never paste them into a chat, email, or public website.

Never save openclaw.json in a publicly shared folder or upload it to GitHub.

STEP 1 Connect to Your VPS

Your VPS is a computer that lives in a data center. To work on it, you connect over the internet using SSH — think of it as a secure text-based phone call to your server. You type commands on your computer and they run on the server.

Choose one option below. You only need one.

Option	Best For
Option A — Terminal	Quickest. Works on any computer with a terminal app.
Option B — VS Code	More comfortable. Shows a file explorer and highlights config errors in real time.

Option A — Connect Using a Terminal

A terminal is a text window where you type commands. On Mac it's called Terminal. On Windows, use PowerShell or Windows Terminal.

1. Open a terminal on your computer.
2. Type the command below, replacing the example values with your own:

```
ssh your_username@your_server_address
```

Example — if your username is root and your server's address is 203.0.113.50:

```
ssh root@203.0.113.50
```



Good to Know

If you use a key file to log in (instead of a password), use this format:

```
ssh -i /path/to/your/keyfile.pem your_username@your_server_address
```

3. Press Enter. Type your password when prompted. You won't see letters appear as you type — that's normal and expected.



Check — You Should See This

Your terminal prompt changes to show your server's name or IP. Everything you type now runs on the server, not your own computer.

To open and edit the config file:

```
nano ~/.openclaw/openclaw.json
```

How to Use nano (the Terminal Text Editor)

nano is a simple editor that works inside the terminal. Use your keyboard only — your mouse won't move the cursor.

Action	Keys to Press
Save the file	Ctrl + O, then press Enter to confirm
Exit nano	Ctrl + X
Save and exit	Ctrl + O → Enter → Ctrl + X
Undo a change	Alt + U
Jump to a line number	Ctrl + _ (underscore), then type the number

Option B — Connect Using VS Code

VS Code is a free code editor by Microsoft. With the Remote - SSH extension, you edit files on your server as if they were on your own computer — with color-coded text, live error highlighting, and a file explorer.

Part 1: Install the Extension

4. Open VS Code on your computer.
5. Press **Ctrl + Shift + X** (Windows/Linux) or **Cmd + Shift + X** (Mac) to open the Extensions panel.
6. Search for **Remote - SSH**.
7. Click **Install** on the result published by Microsoft (blue verified checkmark).

- After installation, a **><** icon appears in the very bottom-left corner of VS Code.

Good to Know

Don't see the **><** icon? Close VS Code completely and reopen it. It lives in the very bottom-left of the window.

Part 2: Add Your Server as a Host

- Click the **><** icon → **Connect to Host...**
- Select **Add New SSH Host...**
- Type your SSH command (same format as Option A):

```
ssh your_username@your_server_address
```

- Press Enter. When asked which config file to update, choose `~/.ssh/config`.

Part 3: Connect

- Click **><** → **Connect to Host...**
- Click your server's IP address from the list.
- A new VS Code window opens. The first time, it installs a small helper on your server — takes about 30–60 seconds.
- When asked for the remote platform, select **Linux**.
- Enter your password or approve the key prompt.

Check — You Should See This

The bottom-left corner turns green and shows your server's IP. Any file you save now goes directly to the server.

Part 4: Open Your OpenClaw Folder

- Press **Ctrl + Shift + E** (or **Cmd + Shift + E**) to open the Explorer sidebar.
- Click **Open Folder**.
- Type the path that matches your setup:

Your Setup	Path to Enter
Default installation	/root/.openclaw
Non-root user	/home/your_username/.openclaw
Docker container	/docker/<container-id>/data/.openclaw

- Press Enter. If asked **"Do you trust the authors?"** — click **Yes, I trust the authors**.

**Good to Know**

Open a terminal inside VS Code by pressing Ctrl + ` (the backtick key, top-left of most keyboards). This terminal connects directly to your server.

VS Code underlines JSON errors in red as you type. Press Ctrl+Shift+M (Cmd+Shift+M on Mac) to see all errors at once.

STEP 2 Understand Your Config File

OpenClaw stores all its settings in a single file called `openclaw.json`. It's a structured text file — a series of labeled sections, each containing settings for a specific part of the app.

**Important — Read Before Continuing**

You will never replace this entire file.

Each step in this guide makes a small, targeted edit — adding or updating one specific section.

Everything else in the file stays exactly as-is.

View the File Before Editing

Take a look at what's already there. In your terminal:

```
cat ~/.openclaw/openclaw.json
```

You'll see something like this (abbreviated for readability):

```
{
  "meta":      { "lastTouchedAt": "...", ... },
  "update":    { "channel": "stable", ... },
  "browser":   { "headless": true, ... },
  "models":    { "mode": "merge" },
  "agents":    { "defaults": { ... }, "list": [ ... ] },
  "commands":  { ... },
  "hooks":     { ... },
  "channels":  { ... },
  "gateway":   { ... },
  "plugins":   { ... }
}
```

Each word before the colon ("meta", "agents", "plugins", etc.) is a top-level section. This guide makes targeted edits inside the existing "agents" section only. You won't be adding any new top-level sections.

STEP 3 Add API Keys and Configure Model Routing

This step makes two edits to `openclaw.json`:

22. Add an **"env"** section at the top — this is where your API keys live.
23. Update the **"agents"** section — to set Claude Haiku 4.5 as the default model and register all the other models.



Good to Know

The routing strategy this guide sets up:

- Default model: Claude Haiku 4.5 — fast and affordable, handles most tasks
- For complex tasks: switch to Claude Sonnet 4.5 manually when needed
- Fallback models: registered so you can switch to them if Anthropic is unavailable

Edit 1 of 2 — Add Your API Keys

Open the config file:

```
nano ~/.openclaw/openclaw.json
```

Look at the very beginning of the file. It starts like:

```
{, "  "meta": {
```

Add the "env" section above "meta" so the top of your file looks like this:

```
{
  "env": {
    "ANTHROPIC_API_KEY": "sk-ant-YOUR_KEY_HERE",
    "OPENAI_API_KEY":    "sk-YOUR_OPENAI_KEY_HERE",
    "GOOGLE_API_KEY":    "YOUR_GEMINI_KEY_HERE",
    "DEEPSEEK_API_KEY":  "YOUR_DEEPSEEK_KEY_HERE"
  },
  "meta": {
```



Important — Read Before Continuing

Replace each placeholder with your real API key. For example, replace `sk-ant-YOUR_KEY_HERE` with your actual Anthropic key.

Only include providers you have keys for. If you don't have an OpenAI key, delete that line. Also remove its model entries in Edit 2.

If an "env" section already exists in your file, don't create a second one — just add your missing key lines inside the existing one.

Never upload or share this file. Your API keys are inside it.

Edit 2 of 2 — Update the Agents Section

Now find this existing block in your file, inside the "agents" section under "defaults":

```
"defaults": {
  "model": {
    "primary": "Claude Opus 4.6"
  },
  "models": {
    "anthropic/claude-opus-4-6": {
      "alias": "Claude Opus 4.6"
    },
    "anthropic/claude-sonnet-4-6": {
      "alias": "Claude Sonnet 4.6"
    },
    "anthropic/claude-haiku-4-5": {
      "alias": "Claude Haiku 4.5"
    }
  }
},
```

Replace that entire block with this (leave "list": [...] and everything else in "agents" untouched):

```
"defaults": {
  "model": {
    "primary": "anthropic/claude-haiku-4-5"
  },
  "models": {
    "anthropic/claude-opus-4-6": { "alias": "opus" },
    "anthropic/claude-sonnet-4-6": { "alias": "sonnet" },
    "anthropic/claude-haiku-4-5": { "alias": "haiku" },
    "openai/gpt-5-mini": { "alias": "gpt-5-mini" },
    "openai/gpt-5.1": { "alias": "gpt-5.1" },
    "google/gemini-2.0-flash": { "alias": "gemini-flash" },
    "google/gemini-2.0-pro": { "alias": "gemini-pro" },
    "deepseek/deepseek-chat": { "alias": "deepseek" },
    "deepseek/deepseek-reasoner": { "alias": "deepseek-r1" }
  }
},
```

Important — Read Before Continuing

The "primary" value must be the full model path (e.g. "anthropic/claude-haiku-4-5"), not an alias. Using an alias causes an 'Invalid input' error on startup.

Each model entry supports "alias" and optionally "params". Cache is configured via "params": { "cacheRetention": "short" } — not as a bare "cache" field. Fields like tier, role, cache_ttl, and fallback_chain are invalid and will cause errors.

If you skipped a provider in Edit 1 (no API key), remove its model entries here too.

The comma at the end of the closing } of "defaults" must stay — the "list" block that follows it requires it.

**Good to Know**

How primary and aliases work: "primary" takes the full model path (like "anthropic/claude-haiku-4-5"). The "alias" is a short nickname you can use to refer to that model in SOUL.md instructions — keep aliases short and lowercase.

STEP 4 Add Routing Rules to SOUL.md

The config file tells OpenClaw which models exist. SOUL.md tells your agent when to use each one. Think of SOUL.md as a set of standing instructions the agent reads before every task.

Open the file:

```
nano ~/.openclaw/workspace/SOUL.md
```

Paste this block at the very top of the file — before any content already there:

```
=====
MODEL ROUTING RULES — READ BEFORE EVERY TASK
=====

DEFAULT MODEL: Always start with "haiku" (Claude Haiku 4.5).

SWITCH TO "sonnet" (Claude Sonnet 4.5) only when the task requires:
- Designing or reviewing system architecture
- Reviewing production code
- Security analysis or vulnerability scanning
- Debugging after 2 failed attempts with Haiku
- A major decision affecting multiple projects

IF ANTHROPIC IS UNAVAILABLE, switch to these models in order:
1. "gpt-5-mini"      (OpenAI)      — for standard tasks
2. "gemini-flash"    (Google)      — for standard tasks
3. "deepseek"        (DeepSeek)    — for standard tasks

For complex tasks that need Sonnet-level capability:
1. "gpt-5.1"         (OpenAI)
2. "gemini-pro"      (Google)
3. "deepseek-r1"     (DeepSeek)

NEVER switch models mid-task unless you hit a rate limit error.
NEVER use a premium model for: writing/reading files, simple questions,
status updates, formatting, or anything Haiku handles in one attempt.

=====
```

Save and exit: Ctrl + O → Enter → Ctrl + X

STEP 5 Configure Session Initialization

By default, OpenClaw loads your full conversation history on every message. That wastes tokens (which cost money). This step limits what the agent loads at the start of each session.

Open SOUL.md again:

```
nano ~/.openclaw/workspace/SOUL.md
```

Scroll to the bottom (below the routing rules from Step 4). Paste this block:

```
=====
SESSION INITIALIZATION - LOAD LIMITS
=====

AT THE START OF EVERY SESSION, load ONLY:
- SOUL.md                (core identity and principles)
- USER.md                (user preferences and profile)
- memory/YYYY-MM-DD.md   (today's memory file, if it exists)

DO NOT automatically load:
- Full conversation history
- MEMORY.md (the full memory file)
- Sessions or logs from previous days
- Tool outputs from past sessions

WHEN THE USER ASKS ABOUT PAST CONTEXT:
1. Run: memory_search("relevant keyword")
2. If found, run: memory_get("entry id")
3. Return only the relevant snippet - do not load the whole file

AT THE END OF EVERY SESSION:
- Write a summary to memory/YYYY-MM-DD.md
- Keep it under 500 words
- Format: bullet points only

=====
```

Save and exit: Ctrl + O → Enter → Ctrl + X

STEP 6 Set Up Local Heartbeat with Ollama

A heartbeat is a scheduled check-in that OpenClaw sends to an AI model on a timer. By default this would use your paid API credits. This step routes those check-ins to Ollama — a free AI that runs locally on your server — so they never cost you anything.

Part A: Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Verify the install:

```
ollama --version
```

✓ **Check — You Should See This**

You see a version number, like: ollama version 0.x.x

Part B: Start the Ollama Service

Run these three commands one at a time:

```
systemctl start ollama
systemctl enable ollama
systemctl status ollama
```

✓ **Check — You Should See This**

The last command shows: active (running)

Part C: Download the Local AI Model (~2 GB)

This takes a few minutes depending on your connection speed.

```
ollama pull llama3.2:3b
```

Test that it's working:

```
ollama run llama3.2:3b "Respond with only the word OK"
```

✓ **Check — You Should See This**

The model replies with: OK

Press Ctrl + D to exit the model prompt.

Part D: Configure the Heartbeat in openclaw.json

Open your config:

```
nano ~/.openclaw/openclaw.json
```

Find the "agents" section. Inside it, locate the "defaults" block. Add a "heartbeat" key directly inside "defaults" (alongside "model" and "models"):



Good to Know

The heartbeat setting lives inside agents → defaults, not at the top level of the file.

If your config does not already have a heartbeat key inside defaults, add it there.

Do not add heartbeat as a new top-level section — OpenClaw will reject it.

Find this part of "defaults" (you already edited this in Step 3):

```
"defaults": {  
  "model": {  
    "primary": "anthropic/claude-haiku-4-5"  
  },  
  "models": {  
    ...  
  }  
},
```

Add the heartbeat block inside defaults, after the "models" section:

```
"defaults": {  
  "model": {  
    "primary": "anthropic/claude-haiku-4-5"  
  },  
  "models": {  
    ...  
  },  
  "heartbeat": {  
    "every": "1h",  
    "model": "ollama/llama3.2:3b",  
    "session": "main",  
    "target": "slack",  
    "prompt": "Briefly check: any blockers, pending tasks, or reminders?"  
  }  
},
```



Good to Know

Change "slack" to the platform where you want heartbeat messages delivered.

The comma after the closing } of "heartbeat" is required because "defaults" continues after it.

STEP 7 Set Spending Rules

This step adds budget guidelines to SOUL.md — instructions your agent follows to pace its API usage and notify you before costs get out of hand. These are plain-text instructions to the agent, not config file changes.

Open SOUL.md:

```
nano ~/.openclaw/workspace/SOUL.md
```

Scroll to the bottom and paste this after everything already there:

```
=====
RATE LIMITS & BUDGET RULES
=====

API CALL PACING:
- Minimum 5 seconds between consecutive API calls
- Minimum 10 seconds between web search requests
- After 5 web searches in a row: pause for 2 full minutes

TASK BATCHING:
- Group similar tasks into a single message when possible
- Never make multiple separate API calls when one will do

DAILY SPEND TARGET: $5.00
- At $3.75 (75%): Notify the user before continuing
- At $5.00 (100%): Stop and ask the user to confirm before proceeding

MONTHLY SPEND TARGET: $150.00
- At $112.50 (75%): Send a summary and ask whether to continue
- At $150.00 (100%): Halt all non-essential operations

IF YOU HIT A RATE LIMIT ERROR:
1. Switch to the next available model in the fallback list
2. Note which model you switched to
3. Retry the same task on the new model
4. Tell the user what happened at the end of the session

=====
```

Save and exit: Ctrl + O → Enter → Ctrl + X

STEP 8 Enable Prompt Caching

Prompt caching lets the AI provider reuse your system prompt (SOUL.md, USER.md) across messages in a session. Instead of processing those files from scratch on every single message, the provider stores them after the first message and reads from that cache on every message after — at roughly 90% lower cost for the cached portion.



Good to Know

Caching is configured through the "params" field on individual model entries in openclaw.json. There are no separate cache blocks or top-level cache settings — it all lives on the model.

How to Set Cache Retention on a Model

Open the config file:

```
nano ~/.openclaw/openclaw.json
```

Find the "models" block inside agents → defaults. Add a "params" field with "cacheRetention" to each model you want to cache:

```
"defaults": {
  "model": {
    "primary": "anthropic/claude-haiku-4-5"
  },
  "models": {
    "anthropic/claude-opus-4-6": {
      "alias": "opus",
      "params": { "cacheRetention": "long" }
    },
    "anthropic/claude-sonnet-4-6": {
      "alias": "sonnet",
      "params": { "cacheRetention": "short" }
    },
    "anthropic/claude-haiku-4-5": {
      "alias": "haiku"
    },
    "openai/gpt-5-mini": { "alias": "gpt-5-mini" },
    "openai/gpt-5-1": { "alias": "gpt-5.1" },
    "google/gemini-2.0-flash": { "alias": "gemini-flash" },
    "google/gemini-2.0-pro": { "alias": "gemini-pro" },
    "deepseek/deepseek-chat": { "alias": "deepseek" },
    "deepseek/deepseek-reasoner": { "alias": "deepseek-r1" }
  }
},
```

cacheRetention value	What it means
"none"	Caching disabled for this model — every message pays full price
"short"	Cache held for approximately 5 minutes of inactivity
"long"	Cache held for approximately 1 hour of inactivity



Good to Know

Haiku has no cacheRetention set at all — it's cheap enough that caching overhead is not worth it.

Opus and Sonnet benefit most from caching because they are expensive models with large system prompts.

cacheRetention only works with Anthropic models (direct API or Amazon Bedrock). For OpenAI, Google, and DeepSeek models, the setting has no effect and can be left off.

Keep the Cache Warm with Heartbeat

If there is a gap of more than an hour between messages, the cache expires and the next message has to rebuild it at full cost. You can prevent this by configuring a heartbeat — a scheduled ping OpenClaw sends automatically to keep the cache alive.

Add the heartbeat setting inside agents → defaults, alongside "model" and "models":

```
"defaults": {
  "model": {
    "primary": "anthropic/claude-haiku-4-5"
  },
  "heartbeat": {
    "every": "55m"
  },
  "models": {
    ...
  }
},
```



Good to Know

55 minutes is the recommended interval — it fires just before the 1-hour cache TTL expires, keeping the cache continuously warm.

You can also set a heartbeat per individual agent. Add the same "heartbeat": { "every": "55m" } block inside the specific agent entry under agents.list[].

Enable Context Pruning (Recommended)

As a session grows longer, old tool outputs and past messages accumulate and get sent with every new message — even when they're no longer relevant. Context pruning removes stale content after the cache TTL window so it doesn't bloat your costs.

Add contextPruning inside agents → defaults:

```
"defaults": {
  "model": { ... },
  "heartbeat": { "every": "55m" },
  "contextPruning": {
    "mode": "cache-ttl",
    "ttl": "1h"
  },
  "models": { ... }
},
```

Monitor Cache Performance

OpenClaw can write a detailed cache trace log so you can see exactly how many tokens are being served from cache vs. being written fresh each session.

Add this diagnostics block to the top level of `openclaw.json` (as a sibling of "agents", "commands", etc.):

```
"diagnostics": {
  "cacheTrace": {
    "enabled": true
  }
},
```

Cache trace logs are written to:

```
~/.openclaw/logs/cache-trace.jsonl
```



Good to Know

Each line in the log is a JSON event. Look for "cacheRead" and "cacheWrite" values in the events — cacheRead tells you how many tokens were served cheaply from cache, cacheWrite tells you how many were charged at full price to build the cache.

You can also check cache performance per session by running: `openclaw shell` → then: `/usage full`



Important — Read Before Continuing

High cacheWrite on most turns (not just the first) usually means your system prompt is changing between messages. Check that nothing is modifying `SOUL.md` or `USER.md` during a session — any change to those files resets the cache.



Check — You Should See This

Setup complete! With model routing, session limits, caching, and heartbeat keep-warm all configured, routine costs should be a fraction of the unconfigured default.

Troubleshooting — Common Problems and Fixes

Find your error message below and follow the fix.

"Invalid config" or validation errors on startup

Check your config against these rules:

Rule	Detail
"primary" must be a full model path	Use "anthropic/claude-haiku-4-5", not an alias. Using an alias causes an 'Invalid input' error.
Cache goes under model params	Correct: "params": { "cacheRetention": "short" } inside the model entry. Wrong: top-level cache block, or "cache": true directly on the model.

heartbeat must be inside defaults	Do not add heartbeat as a top-level section. It belongs inside agents → defaults as "heartbeat": { "every": "55m" }.
diagnostics is a top-level section	The diagnostics block is a sibling of "agents" and "commands" — not nested inside agents.
These fields are not valid anywhere	tier, role, cache (bare boolean), cache_ttl, fallback_chain — remove these if present.

API key not being picked up

OpenClaw only reads the config on startup. Restart OpenClaw after any change to your API keys.

"ollama: command not found"

Your shell hasn't loaded the updated PATH. Run:

```
source ~/.bashrc && ollama --version
```

If it still doesn't work:

```
sudo reboot
```

"ollama pull" fails or stalls

Check available disk space (you need at least 3 GB free):

```
df -h
```

Ollama service won't start

View error logs:

```
journalctl -u ollama -n 50
```

If your server has very low RAM, run Ollama manually instead:

```
ollama serve &
```

VS Code Remote SSH won't connect

First confirm plain SSH works from your terminal:

```
ssh your_username@your_server_address
```

If SSH works but VS Code still won't connect, remove the stale VS Code server helper:

24. Click the green >< icon
25. Select **Remote-SSH: Kill VS Code Server on Host...**
26. Select your server, then try reconnecting

Still failing? Open `~/.ssh/config` and check that the hostname and username are correct.

 Important — Read Before Continuing

Security reminder: Never commit `openclaw.json` to a public code repository. It contains your API keys.

Add it to your gitignore to prevent accidents:

```
echo ".openclaw/" >> ~/.gitignore
```

— *End of Guide* —